Introduction

Behavioral design patterns are centered on how objects communicate and interact with each other, helping define the flow of control within a system. These patterns simplify the complex logic involved in communication while promoting loose coupling between objects.

Imagine a vending machine that changes its behavior based on the coins inserted — when you insert enough money, the machine gives you a snack and when you haven't inserted enough, it asks for more. This dynamic change in behavior depending on the machine's state is what the State Pattern addresses.

Let's understand explore **State Pattern** in detail in the upcoming sections.

State Pattern

Formal Definition

The **State Pattern** is a behavioral design pattern that **encapsulates state-specific behavior into separate classes** and delegates the behavior to the appropriate state object. This allows the object to change its behavior without altering the underlying code.

This pattern makes it easy to manage state transitions by isolating state-specific behavior into distinct classes. It helps achieve **loose coupling** by ensuring that each state class is independent and can evolve without affecting others.

Real-Life Analogy

Consider a **food delivery app**. As an order progresses, its state changes through multiple stages:

- The order is placed.
- The order is being prepared.
- A delivery partner is assigned.
- The order is picked up.
- The order is out for delivery.
- Finally, the order is delivered.

At each stage, the app behaves differently:

- In the "Order Placed" state, you can cancel the order.
- In the "Order Preparing" state, you can track the preparation status.
- In the "Delivery Partner Assigned" state, you can see the details of the assigned driver.
- And so on until the order is delivered.

Each of these states represents a distinct phase, and the app's behavior changes based on which state the order is in. The **State Pattern** manages these transitions seamlessly, with each state class controlling the behavior for that phase. It also follows **Open Closed Principle (OCP)**, as states can be added without modifying the existing code.

Let's now understand the working of **State Pattern** through the help of a problem statement.

Understanding The Problem

Let's assume we are building a food delivery app, and we need to manage the different states of an order. The order can transition between multiple states, such as placed, preparing, out for delivery, and delivered.

Below is a simplified version of how we might implement this without using the State Pattern:

```
#include <bits/stdc++.h>

using namespace std;


class Order {

private:

   string state;


public:

   // Constructor initializes the state to ORDER_PLACED

   Order() {

      this->state = "ORDER_PLACED";

   }


   // Method to cancel the order

   // only allows cancellation if in ORDER_PLACED or PREPARING states

   void cancelOrder() {

      if (state == "ORDER_PLACED" || state == "PREPARING") {
```

```
        state = "CANCELLED";
        cout << "Order has been cancelled." << endl;
    } else {
        cout << "Cannot cancel the order now." << endl;
    }
}

// Method to move the order to the next state based on its current state
void nextState() {
    if (state == "ORDER_PLACED") {
        state = "PREPARING";
    } else if (state == "PREPARING") {
        state = "OUT_FOR_DELIVERY";
    } else if (state == "OUT_FOR_DELIVERY") {
        state = "DELIVERED";
    } else {
        cout << "No next state from: " << state << endl;
        return;
    }
    cout << "Order moved to: " << state << endl;
}
```

```
    // Getter for the state
    string getState() {
        return state;
    }
};

int main() {
    Order order;

    // Display initial state
    cout << "Initial State: " << order.getState() << endl;

    // Moving through states
    order.nextState(); // ORDER_PLACED -> PREPARING
    order.nextState(); // PREPARING -> OUT_FOR_DELIVERY
    order.nextState(); // OUT_FOR_DELIVERY -> DELIVERED

    // Attempting to cancel an order after it is out for delivery
    order.cancelOrder(); // Should not allow cancellation
```

```
    // Display final state
    cout << "Final State: " << order.getState() << endl;

    return 0;
}

import java.util.*;

class Order {
    private String state;

    // Constructor initializes the state to ORDER_PLACED
    public Order() {
        this.state = "ORDER_PLACED";
    }

    // Method to cancel the order
    // only allows cancellation if in ORDER_PLACED or PREPARING states
    public void cancelOrder() {
        if (state.equals("ORDER_PLACED") || state.equals("PREPARING")) {
            state = "CANCELLED";
```

```
        System.out.println("Order has been cancelled.");
    } else {
        System.out.println("Cannot cancel the order now.");
    }
}

// Method to move the order to the next state based on its current state
public void nextState() {
    switch (state) {
        case "ORDER_PLACED":
            state = "PREPARING";
            break;
        case "PREPARING":
            state = "OUT_FOR_DELIVERY";
            break;
        case "OUT_FOR_DELIVERY":
            state = "DELIVERED";
            break;
        default:
            System.out.println("No next state from: " + state);
            return;
```

```
        }
        System.out.println("Order moved to: " + state);
    }

    // Getter for the state
    public String getState() {
        return state;
    }
}

class Main {
    // Main method to test the order flow
    public static void main(String[] args) {
        Order order = new Order();

        // Display initial state
        System.out.println("Initial State: " + order.getState());

        // Moving through states
        order.nextState(); // ORDER_PLACED -> PREPARING
        order.nextState(); // PREPARING -> OUT_FOR_DELIVERY
```

```
        order.nextState(); // OUT_FOR_DELIVERY -> DELIVERED

        // Attempting to cancel an order after it is out for delivery
        order.cancelOrder(); // Should not allow cancellation

        // Display final state
        System.out.println("Final State: " + order.getState());
    }
}

class Order:
    def __init__(self):
        # Constructor initializes the state to ORDER_PLACED
        self.state = "ORDER_PLACED"

    # Method to cancel the order
    # only allows cancellation if in ORDER_PLACED or PREPARING states
    def cancelOrder(self):
        if self.state == "ORDER_PLACED" or self.state == "PREPARING":
            self.state = "CANCELLED"
            print("Order has been cancelled.")
```

```
    else:
        print("Cannot cancel the order now.")

# Method to move the order to the next state based on its current state
def nextState(self):
    if self.state == "ORDER_PLACED":
        self.state = "PREPARING"
    elif self.state == "PREPARING":
        self.state = "OUT_FOR_DELIVERY"
    elif self.state == "OUT_FOR_DELIVERY":
        self.state = "DELIVERED"
    else:
        print(f"No next state from: {self.state}")
        return
    print(f"Order moved to: {self.state}")

# Getter for the state
def getState(self):
    return self.state
```

```
# Main function to test the order flow
if __name__ == "__main__":
    order = Order()

    # Display initial state
    print("Initial State:", order.getState())

    # Moving through states
    order.nextState()  # ORDER_PLACED -> PREPARING
    order.nextState()  # PREPARING -> OUT_FOR_DELIVERY
    order.nextState()  # OUT_FOR_DELIVERY -> DELIVERED

    # Attempting to cancel an order after it is out for delivery
    order.cancelOrder()  # Should not allow cancellation

    # Display final state
    print("Final State:", order.getState())
```

The above code works fine, but there are some critical issues in the code.

Issues In The Code

- **State Transition Management:**
  The state transitions are hardcoded in the `nextState()` method using a switch statement. This approach becomes cumbersome if new states need to be added.
- **Lack of Encapsulation:**
  The state transition logic and cancel behavior are directly handled within the Order class. This violates the Single Responsibility Principle by combining multiple responsibilities within a single class.
- **Code Duplication:**
  The logic for the `cancelOrder()` and `nextState()` methods could lead to duplicate logic if more states and actions are added.
- **Missing Flexibility for Future Changes:**
  Adding new states or changing existing behaviors can be error-prone and cumbersome, as the Order class needs to be updated each time.

Solution

The previous code can be improved using the **State Pattern**. Below is the refactored code implementing the **State Pattern**:

#include <iostream>

#include <string>

using namespace std;

//================================

// OrderState class defines the behavior of the order states

//================================

```
class OrderState {
public:
   virtual void next(class OrderContext* context) = 0; // Move to the next state
   virtual void cancel(class OrderContext* context) = 0; // Cancel the order
   virtual string getStateName() = 0; // Get the name of the state
};

//================================
// Forward declaration of the OrderContext class
//================================
class OrderContext;

//================================
// Concrete states for each stage of the order
//================================

// OrderPlacedState handles the behavior when the order is placed
class OrderPlacedState : public OrderState {
public:
   void next(OrderContext* context) override;
```

```
    void cancel(OrderContext* context) override;
    string getStateName() override;
};

// PreparingState handles the behavior when the order is being prepared
class PreparingState : public OrderState {
public:
    void next(OrderContext* context) override;
    void cancel(OrderContext* context) override;
    string getStateName() override;
};

// OutForDeliveryState handles the behavior when the order is out for delivery
class OutForDeliveryState : public OrderState {
public:
    void next(OrderContext* context) override;
    void cancel(OrderContext* context) override;
    string getStateName() override;
};

// DeliveredState handles the behavior when the order is delivered
```

```
class DeliveredState : public OrderState {
public:
    void next(OrderContext* context) override;
    void cancel(OrderContext* context) override;
    string getStateName() override;
};

// CancelledState handles the behavior when the order is cancelled
class CancelledState : public OrderState {
public:
    void next(OrderContext* context) override;
    void cancel(OrderContext* context) override;
    string getStateName() override;
};

//================================
// OrderContext class manages the current state of the order
//================================
class OrderContext {
private:
    OrderState* currentState;
```

```
public:
    // Constructor initializes the state to ORDER_PLACED
    OrderContext();

    // Method to set a new state for the order
    void setState(OrderState* state);

    // Method to move the order to the next state
    void next();

    // Method to cancel the order
    void cancel();

    // Method to get the current state of the order
    string getCurrentState();
};

//================================
// OrderContext constructor initializes the default state to OrderPlacedState
//================================
```

```
OrderContext::OrderContext() {
    currentState = new OrderPlacedState(); // default state
}

// Method to set a new state for the order
void OrderContext::setState(OrderState* state) {
    currentState = state;
}

// Method to move the order to the next state
void OrderContext::next() {
    currentState->next(this);
}

// Method to cancel the order
void OrderContext::cancel() {
    currentState->cancel(this);
}

// Method to get the current state of the order
string OrderContext::getCurrentState() {
```

```
    return currentState->getStateName();
}

//================================
// OrderPlacedState handles the behavior when the order is placed
//================================
void OrderPlacedState::next(OrderContext* context) {
    context->setState(new PreparingState());
    cout << "Order is now being prepared." << endl;
}

void OrderPlacedState::cancel(OrderContext* context) {
    context->setState(new CancelledState());
    cout << "Order has been cancelled." << endl;
}

string OrderPlacedState::getStateName() {
    return "ORDER_PLACED";
}

//================================
```

```
// PreparingState handles the behavior when the order is being prepared
//================================
void PreparingState::next(OrderContext* context) {
    context->setState(new OutForDeliveryState());
    cout << "Order is out for delivery." << endl;
}

void PreparingState::cancel(OrderContext* context) {
    context->setState(new CancelledState());
    cout << "Order has been cancelled." << endl;
}

string PreparingState::getStateName() {
    return "PREPARING";
}

//================================
// OutForDeliveryState handles the behavior when the order is out for delivery
//================================
void OutForDeliveryState::next(OrderContext* context) {
    context->setState(new DeliveredState());
```

```
    cout << "Order has been delivered." << endl;
}

void OutForDeliveryState::cancel(OrderContext* context) {
    cout << "Cannot cancel. Order is out for delivery." << endl;
}

string OutForDeliveryState::getStateName() {
    return "OUT_FOR_DELIVERY";
}

//================================
// DeliveredState handles the behavior when the order is delivered
//================================
void DeliveredState::next(OrderContext* context) {
    cout << "Order is already delivered." << endl;
}

void DeliveredState::cancel(OrderContext* context) {
    cout << "Cannot cancel a delivered order." << endl;
}
```

```
string DeliveredState::getStateName() {
    return "DELIVERED";
}

//================================
// CancelledState handles the behavior when the order is cancelled
//================================
void CancelledState::next(OrderContext* context) {
    cout << "Cancelled order cannot move to next state." << endl;
}

void CancelledState::cancel(OrderContext* context) {
    cout << "Order is already cancelled." << endl;
}

string CancelledState::getStateName() {
    return "CANCELLED";
}

//================================
```

```
// Main function to demonstrate the state transitions
//================================
int main() {
    OrderContext order;

    // Display initial state
    cout << "Current State: " << order.getCurrentState() << endl;

    // Moving through states
    order.next();  // ORDER_PLACED -> PREPARING
    order.next();  // PREPARING -> OUT_FOR_DELIVERY
    order.cancel(); // Should fail, as order is out for delivery
    order.next();  // OUT_FOR_DELIVERY -> DELIVERED
    order.cancel(); // Should fail, as order is delivered

    // Display final state
    cout << "Final State: " << order.getCurrentState() << endl;

    return 0;
}
```

```
import java.util.*;

// OrderContext class manages the current state of the order
class OrderContext {
    private OrderState currentState;

    // Constructor initializes the state to ORDER_PLACED
    public OrderContext() {
        this.currentState = new OrderPlacedState(); // default state
    }

    // Method to set a new state for the order
    public void setState(OrderState state) {
        this.currentState = state;
    }

    // Method to move the order to the next state
    public void next() {
        currentState.next(this);
    }
```

```
    // Method to cancel the order
    public void cancel() {
        currentState.cancel(this);
    }

    // Method to get the current state of the order
    public String getCurrentState() {
        return currentState.getStateName();
    }
}

// OrderState interface defines the behavior of the order states
interface OrderState {
    void next(OrderContext context); // Move to the next state
    void cancel(OrderContext context); // Cancel the order
    String getStateName(); // Get the name of the state
}

// Concrete states for each stage of the order

// OrderPlacedState handles the behavior when the order is placed
```

```
class OrderPlacedState implements OrderState {
    public void next(OrderContext context) {
        context.setState(new PreparingState());
        System.out.println("Order is now being prepared.");
    }

    public void cancel(OrderContext context) {
        context.setState(new CancelledState());
        System.out.println("Order has been cancelled.");
    }

    public String getStateName() {
        return "ORDER_PLACED";
    }
}

// PreparingState handles the behavior when the order is being prepared
class PreparingState implements OrderState {
    public void next(OrderContext context) {
        context.setState(new OutForDeliveryState());
        System.out.println("Order is out for delivery.");
```

```
    }

    public void cancel(OrderContext context) {
        context.setState(new CancelledState());
        System.out.println("Order has been cancelled.");
    }

    public String getStateName() {
        return "PREPARING";
    }
}

// OutForDeliveryState handles the behavior when the order is out for delivery
class OutForDeliveryState implements OrderState {
    public void next(OrderContext context) {
        context.setState(new DeliveredState());
        System.out.println("Order has been delivered.");
    }

    public void cancel(OrderContext context) {
        System.out.println("Cannot cancel. Order is out for delivery.");
```

```
    }

    public String getStateName() {
        return "OUT_FOR_DELIVERY";
    }
}

// DeliveredState handles the behavior when the order is delivered
class DeliveredState implements OrderState {
    public void next(OrderContext context) {
        System.out.println("Order is already delivered.");
    }

    public void cancel(OrderContext context) {
        System.out.println("Cannot cancel a delivered order.");
    }

    public String getStateName() {
        return "DELIVERED";
    }
}
```

```
// CancelledState handles the behavior when the order is cancelled
class CancelledState implements OrderState {
    public void next(OrderContext context) {
        System.out.println("Cancelled order cannot move to next state.");
    }

    public void cancel(OrderContext context) {
        System.out.println("Order is already cancelled.");
    }

    public String getStateName() {
        return "CANCELLED";
    }
}

public class Main {
    public static void main(String[] args) {
        OrderContext order = new OrderContext();

        // Display initial state
```

```
        System.out.println("Current State: " + order.getCurrentState());

        // Moving through states
        order.next();  // ORDER_PLACED -> PREPARING
        order.next();  // PREPARING -> OUT_FOR_DELIVERY
        order.cancel(); // Should fail, as order is out for delivery
        order.next();  // OUT_FOR_DELIVERY -> DELIVERED
        order.cancel(); // Should fail, as order is delivered

        // Display final state
        System.out.println("Final State: " + order.getCurrentState());
    }
}

# OrderContext class manages the current state of the order
class OrderContext:
    def __init__(self):
        self.currentState = OrderPlacedState()  # default state

    # Method to set a new state for the order
    def setState(self, state):
```

```
        self.currentState = state

    # Method to move the order to the next state
    def next(self):
        self.currentState.next(self)

    # Method to cancel the order
    def cancel(self):
        self.currentState.cancel(self)

    # Method to get the current state of the order
    def getCurrentState(self):
        return self.currentState.getStateName()


# OrderState interface defines the behavior of the order states
class OrderState:
    def next(self, context):  # Move to the next state
        pass

    def cancel(self, context):  # Cancel the order
```

```
        pass

    def getStateName(self):  # Get the name of the state
        pass



# Concrete states for each stage of the order


# OrderPlacedState handles the behavior when the order is placed
class OrderPlacedState(OrderState):
    def next(self, context):
        context.setState(PreparingState())
        print("Order is now being prepared.")

    def cancel(self, context):
        context.setState(CancelledState())
        print("Order has been cancelled.")

    def getStateName(self):
        return "ORDER_PLACED"
```

```
# PreparingState handles the behavior when the order is being prepared
class PreparingState(OrderState):
    def next(self, context):
        context.setState(OutForDeliveryState())
        print("Order is out for delivery.")

    def cancel(self, context):
        context.setState(CancelledState())
        print("Order has been cancelled.")

    def getStateName(self):
        return "PREPARING"

# OutForDeliveryState handles the behavior when the order is out for delivery
class OutForDeliveryState(OrderState):
    def next(self, context):
        context.setState(DeliveredState())
        print("Order has been delivered.")
```

```
    def cancel(self, context):
        print("Cannot cancel. Order is out for delivery.")

    def getStateName(self):
        return "OUT_FOR_DELIVERY"


# DeliveredState handles the behavior when the order is delivered
class DeliveredState(OrderState):
    def next(self, context):
        print("Order is already delivered.")

    def cancel(self, context):
        print("Cannot cancel a delivered order.")

    def getStateName(self):
        return "DELIVERED"


# CancelledState handles the behavior when the order is cancelled
class CancelledState(OrderState):
```

```
    def next(self, context):
        print("Cancelled order cannot move to next state.")

    def cancel(self, context):
        print("Order is already cancelled.")

    def getStateName(self):
        return "CANCELLED"


# Main method to test the order flow
if __name__ == "__main__":
    order = OrderContext()

    # Display initial state
    print("Current State:", order.getCurrentState())

    # Moving through states
    order.next()  # ORDER_PLACED -> PREPARING
    order.next()  # PREPARING -> OUT_FOR_DELIVERY
    order.cancel()  # Should fail, as order is out for delivery
```

```
order.next()  # OUT_FOR_DELIVERY -> DELIVERED
order.cancel()  # Should fail, as order is delivered

# Display final state
print("Final State:", order.getCurrentState())
```

How This Solves The Issues

Issue	Solution with State Pattern
State Transition Management	The transitions are now handled by the respective state classes, making it easy to add new states or modify transitions without altering the `OrderContext` class. Each state handles its own behavior.
Lack of Encapsulation	Each state is now encapsulated in its own class, adhering to the **Single Responsibility Principle** and ensuring better maintainability.
Code Duplication	The behavior for each state is encapsulated in the corresponding class, avoiding duplicate logic for state transitions and cancellations.

Flexibility for Future Changes	New states can be added easily by creating new classes implementing the `OrderState` interface, without modifying existing code. This follows the **Open/Closed Principle**.

When to Use the State Pattern

Here are some scenarios where the State Pattern proves to be highly effective:

- **Object behavior depends on internal state**
  When an object needs to change its behavior based on its internal condition or current phase.
- **Well-defined and finite state transitions**
  When the states and their transitions are clearly structured and limited in number.
- **Avoiding complex if-else or switch-case blocks**
  When you want to eliminate bulky conditional logic that checks for current state and executes accordingly.
- **Need for explicit state transitions**
  When it's important to have clear and maintainable transitions from one state to another.
- **Distinct behavior for each state**
  When each state has its own behavior and rules, making it better to isolate them into separate classes.

Differences Between State and Strategy Pattern

Some users might be confused between the **Strategy Pattern** and the **State Pattern** because both involve the idea of changing behaviors based

on conditions. However, they have distinct purposes and use cases.

Below is a comparison to clarify the differences:

Aspect	State Pattern	Strategy Pattern
Intent	Change behavior based on the object's internal state.	Select an algorithm or behavior at runtime based on content.
Dependency	States can be dependent as you can easily jump from one state to another.	Strategies are completely independent and unaware of each other.
Final Result	It’s about doing different things based on the state, hence the results may vary.	Strategies may end up having the same result, depending on the algorithm selected.
Usage	Workflow models, lifecycle processes, and state machines.	Algorithm selection, formatting, and dynamic behavior handling.

Advantages and Disadvantages of the State Pattern

The **State Pattern** offers several benefits, but also comes with a few trade-offs. Below is an overview of the pros and cons associated with this pattern:

Pros

- **Clear Separation of State Behavior:**
  The State Pattern enables a clean separation of behaviors based on the object's state. Each state is encapsulated in its own class, making it easier to manage and modify individual behaviors without affecting the rest of the system.
- **Easy to Add New States:**
  Adding new states becomes straightforward, as you simply need to create a new class that implements the **OrderState interface**, and it can be integrated with the existing structure with minimal changes.
- **Follows Open/Closed Principle (OCP):**
  The pattern follows the Open/Closed Principle, meaning that it is open for extension but closed for modification. New states can be added without modifying existing code, promoting better maintainability and scalability.
- **Avoids Complex if-else or Switch-case Blocks:**
  With the State Pattern, you can avoid cluttering your code with large and difficult-to-manage conditional statements (such as `if-else` or `switch-case`). Each state class handles its own logic, improving readability and maintainability.

Cons

- **Adds More Classes**
  The introduction of new state classes can make the overall design more complex, as each state behavior requires a dedicated class. This can lead to an increase in the number of classes in the system.
- **Slightly More Complex Initial Setup**
  The initial setup of the State Pattern requires defining multiple state classes and interfaces, which can make the implementation slightly more complex compared to other simpler alternatives.

- **Content Needs to Manage State Transitions**
  The management of state transitions is typically handled by the context or the object holding the states. This introduces an additional layer of complexity in ensuring that state transitions are handled properly throughout the system.
- **Requires Familiarity**
  Developers need to be familiar with the State Pattern, which could be a learning curve for teams or individuals not accustomed to this design. Proper understanding is necessary to implement the pattern effectively.

Real Life Examples

The State Pattern is widely used in various real-life applications where an object's behavior changes depending on its state. Here are three examples of how the State Pattern is implemented in systems:

1. Swiggy (Food Delivery App)

In Swiggy, the order lifecycle is a great example of the State Pattern. As a customer places an order, the order goes through several states:

- Order Placed
- Order Preparing
- Out for Delivery
- Order Delivered
- Order Cancelled

Each of these states has distinct behaviors. For instance, when the order is in the **"Order Placed"** state, the user can only cancel the order. When it's **"Out for Delivery"**, the order cannot be cancelled, and different actions such as live tracking become available. Each state is managed by its own class, implementing the **OrderState** interface. This approach allows easy

transitions between states and simplifies managing order-related actions.

---

2. Uber (Ride-Hailing App)

Uber also utilizes the State Pattern for managing the different stages of a ride. The ride can be in one of several states such as:

- Ride Requested
- Driver Assigned
- Ride Accepted
- Ride In Progress
- Ride Completed
- Ride Cancelled

The behavior of the app varies based on the current state of the ride. For example, if the ride is in the "**Ride Requested**" state, the system allows the user to cancel the ride. Once the ride is "**In Progress**", cancellation is no longer allowed, and the interface changes to show real-time tracking. The state classes handle these transitions, ensuring that the app behaves appropriately at each stage of the ride lifecycle.

---

3. ATM (Automated Teller Machine)

ATMs are another classic example of the State Pattern. The ATM machine can be in one of the following states:

- Idle
- Card Inserted

- PIN Entered
- Transaction In Progress
- Transaction Completed
- Out of Service

Each state dictates the behavior of the machine. For instance, when the machine is in the **"Card Inserted"** state, the user can input their PIN. Once the correct PIN is entered, the machine transitions to the **"Transaction In Progress"** state, where the user can withdraw money or check the balance. The state transition logic ensures that the machine operates in a seamless manner and prevents the user from taking actions that are not allowed in the current state (e.g., entering a PIN after completing a transaction).

Class Diagram